

Prepared in accordance with IEEE 830 / ISO/IEC/IEEE 29148 SRS guidelines

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

for

Online Library Management System

A Responsive Web Platform to Discover, Search and Organize Books

Document Version	1.0
Document Status	Final – For Faculty Evaluation
Prepared For	GLA University
Course / Program	B.Tech (Computer Science & Engineering)
Prepared By	Pushpendra Gupta (Roll No: 22515000106)
Project Guide	Mr. Gautam Mukherjee
Date	20/08/2026

Table of Contents

1. Introduction.....	3
1.1 Purpose.....	3
1.2 Document Conventions.....	3
1.3 Intended Audience and Reading Suggestions.....	3
1.4 Project Scope.....	3
1.5 References.....	4
2. Overall Description.....	4
2.1 Product Perspective.....	4
2.2 Product Features (Summary).....	4
2.3 User Classes and Characteristics.....	4
2.4 Operating Environment.....	5
2.5 Design and Implementation Constraints.....	5
2.6 Assumptions and Dependencies.....	5
3. System Features (Functional Requirements).....	5
3.1 Module: User Registration & Authentication.....	5
3.2 Module: Book Catalog & Search.....	6
3.3 Module: Book Details.....	6
3.4 Module: Personal Reading List (Bookshelf).....	7
3.5 Module: Reviews & Ratings.....	7
3.6 Module: Administrator Portal.....	7
3.7 Module: Notifications.....	8
4. External Interface Requirements.....	8
5. Use Case Summary.....	9
6. Non-Functional Requirements.....	12
7. Software Testing Considerations.....	13
8. Proposed System Architecture.....	15
9. Data Model Overview.....	15
10. Project Timeline & Work Breakdown.....	18
11. Requirements Traceability Matrix.....	18
12. Risk Analysis.....	20
13. Future Enhancements (Out of Current Scope).....	20
14. Glossary.....	21

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the Online Library Management System (OLMS), a responsive web application that enables users to discover, search and organize books, and allows library administrators to maintain the digital book catalog. The document serves as the single source of truth for the student development team, project guide and evaluation committee regarding what the system will do, who will use it, and the constraints under which it must be built. It is intended to support requirement analysis, UI/UX design, database and API planning, software testing, and final project documentation.

1.2 Document Conventions

Functional requirements are uniquely identified using the format FR-XX and are grouped by module. Non-functional requirements use the format NFR-XX. Each requirement is written using the keyword “shall” to indicate a mandatory system capability, consistent with IEEE 830 / ISO/IEC/IEEE 29148 conventions. Priority levels used throughout this document are High (must be implemented for a working system), Medium (important but can be simplified if time is short), and Low (optional / future enhancement).

1.3 Intended Audience and Reading Suggestions

- Students / Development Team – to understand what to build, module by module.
- Project Guide / Faculty Evaluator – to assess completeness, correctness and feasibility of scope.
- UI/UX Designer – to design screens based on Section 4 and the workflows in Section 3.
- Database / Backend Developer – to design schema and APIs based on Section 8 and Section 9.
- Software Tester – to derive test cases directly from the functional requirements in Section 3.

1.4 Project Scope

The Online Library Management System will allow readers to register, browse books by category/genre, search by title, author or keyword, view detailed book information, and maintain a personalized reading list (Want to Read / Currently Reading / Completed). Readers may also rate and review books they have added. Administrators (library staff) will manage the book catalog, categories, and user reviews, and view basic usage reports.

The scope is deliberately kept realistic for a college-level team project. The system manages a digital book catalog and personal reading-organization features — it is not a full-scale physical library circulation system. Physical book issue/return tracking, fine calculation, barcode-based check-in/check-out, e-book reading/downloading, and payment processing are explicitly out of scope for the current version and are listed as future enhancements in Section 13.

1.5 References

- IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications.
- ISO/IEC/IEEE 29148:2018 – Systems and Software Engineering – Requirements Engineering.
- React.js, Node.js, Express.js and MongoDB official documentation.
- Project description / synopsis submitted by the student team.

2. Overall Description

2.1 Product Perspective

The Online Library Management System is a new, self-contained, independent web application. It is not a modification of an existing library system. It follows a standard three-tier web architecture (presentation, application/API and data layer — detailed in Section 8), so it can later be extended with external services such as an ISBN/book-metadata API as the project matures beyond the academic prototype stage.

2.2 Product Features (Summary)

- Reader registration, login and profile management.
- Category/genre-based book browsing with cover thumbnails.
- Keyword search with filters (category, author, rating) and sorting.
- Detailed book information page (description, author, publisher, ISBN, cover).
- Personal reading list with status tracking (Want to Read / Reading / Completed) and favorites.
- Book ratings and reviews by readers.
- Admin dashboard for managing books, categories and moderating reviews.
- Basic analytical reports (most-added books, popular categories, active readers).

2.3 User Classes and Characteristics

User Class	Description	Technical Characteristics
Reader	General user who wants to discover and organize books.	Basic computer/smartphone literacy; primary end-user of search and reading-list features.
Administrator / Librarian	Manages the book catalog, categories and content quality.	Trusted internal user with full catalog-management and reporting access.
Guest (Unauthenticated Visitor)	Visitor who can browse the catalog before registering.	No account required; limited to read-only browsing and search.

2.4 Operating Environment

- Client side: Any modern web browser (Chrome, Firefox, Edge, Safari) on desktop, laptop, tablet or smartphone with an active internet connection.
- Server side: Node.js runtime hosted on a cloud platform (e.g. Render, Railway, AWS/Azure free tier) or a local server for demonstration.
- Database: MongoDB Atlas (cloud) or a local MongoDB instance.

2.5 Design and Implementation Constraints

- Must be developed and demonstrated within a single college semester by a small student team (typically 2–4 members).
- Must primarily use free / open-source technologies to keep project cost at zero.
- Must be responsive and usable on both desktop and mobile screen sizes.
- Cover images are stored as URLs or small uploaded files; large-scale file/CDN storage is treated as an optional enhancement.
- The system assumes a single library/catalog instance (multi-library/multi-tenant support is a future enhancement).

2.6 Assumptions and Dependencies

- Users have basic familiarity with using web forms, search boxes and email.
- The initial book catalog and category list is seeded/set up by the Administrator before readers begin browsing.
- Reliable internet connectivity is available at both client and server ends.
- Email delivery (registration confirmation, password reset) depends on a third-party SMTP service (e.g. Gmail SMTP / SendGrid free tier).

3. System Features (Functional Requirements)

This section lists the functional requirements of the system, grouped by module. Each requirement can be traced directly to a test case (see Section 7) and a screen (see Section 4).

3.1 Module: User Registration & Authentication

Description: Allows readers and administrators to securely create accounts and log in, with role-based access.

ID	Requirement	Priority
FR-01	The system shall allow a new reader to register using name, email and password.	High
FR-02	The system shall validate email format and enforce a minimum password strength on	High

ID	Requirement	Priority
	registration.	
FR-03	The system shall authenticate registered users (Reader, Admin) via email and password login.	High
FR-04	The system shall issue a secure session/token (e.g. JWT) upon successful login.	High
FR-05	The system shall allow a user to reset a forgotten password via a one-time email link/OTP.	Medium
FR-06	The system shall restrict access to module features based on the logged-in user's role (Reader/Admin).	High
FR-07	The system shall allow a logged-in user to log out, invalidating the active session.	Medium
FR-08	The system shall allow a reader to view and update their own profile (name, email, profile photo).	Medium

3.2 Module: Book Catalog & Search

Description: Enables users to discover books through browsing, searching and filtering.

ID	Requirement	Priority
FR-09	The system shall allow a user to browse books by category/genre.	High
FR-10	The system shall allow a user to search books by title, author name or keyword.	High
FR-11	The system shall allow a user to filter search results by category, author or minimum rating.	Medium
FR-12	The system shall display search/browse results as a paginated list showing cover thumbnail, title and author.	High
FR-13	The system shall allow a user to sort book listings by title, newest addition or average rating.	Medium
FR-14	The system shall allow an unauthenticated guest to browse and search the catalog in read-only mode.	Low

3.3 Module: Book Details

Description: Presents complete information about a single book.

ID	Requirement	Priority
FR-15	The system shall display a book's detailed information: title, author(s), description/synopsis, category, publisher, publication year, ISBN and cover image.	High
FR-16	The system shall display a book's average rating and total number of reviews on its details	Medium

ID	Requirement	Priority
	page.	
FR-17	The system shall display the list of individual reviews for a book on its details page.	Medium
FR-18	The system shall provide a one-click action on the book details page to add the book to the reader's personal reading list.	High

3.4 Module: Personal Reading List (Bookshelf)

Description: Lets each reader build and organize a personal collection of books.

ID	Requirement	Priority
FR-19	The system shall allow a reader to add a book to their personal reading list.	High
FR-20	The system shall allow a reader to categorize a reading-list entry as 'Want to Read', 'Currently Reading' or 'Completed'.	High
FR-21	The system shall allow a reader to update the status of a reading-list entry at any time.	High
FR-22	The system shall allow a reader to remove a book from their reading list.	High
FR-23	The system shall allow a reader to mark a book in their reading list as a favorite.	Low
FR-24	The system shall display a dashboard summary of the reader's reading list with counts per status.	Medium

3.5 Module: Reviews & Ratings

Description: Allows readers to share feedback on books.

ID	Requirement	Priority
FR-25	The system shall allow a registered reader to submit a star rating (1–5) and a written review for a book.	Medium
FR-26	The system shall allow a reader to edit or delete their own review.	Medium
FR-27	The system shall recompute and display a book's average rating whenever a review is added, edited or deleted.	Medium

3.6 Module: Administrator Portal

Description: Catalog configuration, oversight and basic reporting.

ID	Requirement	Priority
FR-28	The system shall allow the Admin to add, edit or remove book records in the catalog.	High
FR-29	The system shall allow the Admin to create and manage book categories/genres.	High
FR-30	The system shall allow the Admin to view and moderate (hide/remove) inappropriate reviews.	Medium
FR-31	The system shall allow the Admin to view and manage registered reader accounts.	Medium
FR-32	The system shall allow the Admin to view basic reports: most-added books, popular categories and total active readers.	Medium
FR-33	The system shall allow the Admin to post system-wide announcements (e.g. newly added titles) visible to readers.	Low

3.7 Module: Notifications

ID	Requirement	Priority
FR-34	The system shall send an email confirmation to a reader immediately after successful registration.	Medium
FR-35	The system shall notify a reader by email when their password reset request is processed.	Medium
FR-36	The system shall show an in-app notice to readers when the Admin posts a new announcement.	Low

4. External Interface Requirements

4.1 User Interfaces

- Landing / Home Page – introduction to the platform, featured/new categories, login/register call-to-action.
- Catalog / Browse Page – category filters, search bar, paginated book grid with cover thumbnails.
- Book Details Page – full description, rating summary, reviews list, “Add to Reading List” action.
- My Reading List (Reader Dashboard) – tabs for Want to Read / Currently Reading / Completed, favorites.
- Admin Dashboard – manage books, categories, review moderation, and reports/charts.
- All screens shall follow a consistent, clean, responsive layout (see Section 6.3) using a calm, green-accented colour palette suitable for a reading/library product.

4.2 Hardware Interfaces

No specialized hardware is required. The system runs on standard client devices (desktop/laptop/tablet/smartphone) with a working internet connection, and on any commodity server/cloud instance capable of running Node.js and MongoDB.

4.3 Software Interfaces

Component	Technology
Frontend	React.js (with React Router, Axios, and a component library such as Bootstrap/Tailwind/MUI)
Backend / API	Node.js with Express.js – RESTful JSON APIs
Database	MongoDB (Mongoose ODM) – NoSQL document store
Authentication	JWT (JSON Web Tokens) with bcrypt password hashing
Email Service	SMTP service (e.g. Nodemailer with Gmail/SendGrid) for notifications
Hosting (suggested)	Frontend – Vercel/Netlify; Backend – Render/Railway; Database – MongoDB Atlas

4.4 Communication Interfaces

- All client-server communication shall occur over HTTPS using RESTful JSON APIs.
- Outbound email notifications shall be sent via the SMTP protocol through the configured email service.
- (Future phase) An external book-metadata API (e.g. Google Books / Open Library) may be integrated to auto-fill catalog details.

5. Use Case Summary

The table below summarizes the primary use cases per actor. Detailed step-by-step use-case descriptions (pre-conditions, main flow, alternate flow) can be expanded from this list during design if required by the evaluation format.

Actor	Use Case	Brief Description
Reader	Register / Login	Create an account or sign in to access personalized features.
Reader	Browse / Search Catalog	Discover books by category, keyword or filter.
Reader	View Book Details	See full information, rating and reviews for a book.
Reader	Manage Reading List	Add, update status of, or remove books from a personal list.
Reader	Rate / Review a Book	Share a star rating and written feedback.

Actor	Use Case	Brief Description
Admin	Manage Book Catalog	Add/edit/remove book records.
Admin	Manage Categories	Add/edit/remove genres/categories.
Admin	Moderate Reviews	Hide or remove inappropriate review content.
Admin	View Reports	Analyse catalog usage and popular categories.

5.1 Primary Flow – Add Book to Reading List (Reader)

- 1. Reader logs in.
- 2. Browses or searches for a book of interest.
- 3. Opens the book's details page.
- 4. Clicks “Add to Reading List” and selects a status (Want to Read / Currently Reading / Completed).
- 5. System saves the entry and links it to the reader's account.
- 6. The book now appears on the reader's “My Reading List” dashboard.

5.2 Use Case: Reader Searches and Filters Books

Use Case ID	UC-02
Actor(s)	Reader / Guest
Description	Allows a user to locate books of interest using keyword search combined with category, author or rating filters.
Preconditions	The catalog contains at least one published book record.
Main Flow	1. User opens the Catalog/Browse page. 2. User enters a keyword in the search bar and/or selects a category filter. 3. System queries the catalog and returns matching results. 4. User applies additional filters (author, minimum rating) or changes the sort order. 5. User selects a book from the results to view its details.
Alternate Flow	If no book matches the search/filter combination, the system displays an empty-state message with a suggestion to broaden the search (FR-10, FR-11).
Postcondition	User is presented with a relevant, ranked list of books (FR-09–FR-13).

5.3 Use Case: Reader Submits a Book Review

Use Case ID	UC-03
Actor(s)	Reader

Description	Enables a logged-in reader to rate a book and leave a written review.
Preconditions	Reader is registered and logged in; the target book exists in the catalog.
Main Flow	1. Reader opens a book's details page. 2. Reader selects a star rating (1–5) and enters review text. 3. Reader submits the review. 4. System validates and stores the review, linking it to the reader and the book. 5. System recalculates and displays the book's updated average rating.
Alternate Flow	If the reader has already reviewed the book, the system opens the existing review in edit mode instead of creating a duplicate (FR-26).
Postcondition	The review is visible on the book's details page and included in the average rating (FR-25, FR-27).

5.4 Use Case: Admin Adds a New Book to Catalog

Use Case ID	UC-04
Actor(s)	Administrator
Description	Allows the administrator to add a new book record to the catalog.
Preconditions	Administrator is logged in with valid credentials.
Main Flow	1. Admin navigates to the Manage Books section. 2. Admin clicks “Add New Book” and fills in title, author, category, description, ISBN and cover image. 3. Admin submits the form. 4. System validates required fields and stores the new book record. 5. The new book becomes immediately searchable and browsable by readers.
Alternate Flow	If a required field is missing or the ISBN already exists, the system rejects the submission with a clear validation message (FR-28).
Postcondition	Catalog contains the new, searchable book record (FR-28).

5.5 Use Case: Admin Manages Categories

Use Case ID	UC-05
Actor(s)	Administrator
Description	Maintains the master list of book categories/genres used for browsing and filtering.
Preconditions	Administrator is logged in.
Main Flow	1. Admin opens the Manage Categories section. 2. Admin adds a new category name and description, or edits/removes an existing one. 3. System validates uniqueness of the category name. 4. Updated category list becomes immediately available in the browse/filter menus.

Alternate Flow	If the admin attempts to remove a category that still has books assigned to it, the system warns the admin and requires reassignment or confirmation before deletion (FR-29).
Postcondition	Category master data is up to date across the system (FR-29).

5.6 Use Case: Admin Moderates a Review

Use Case ID	UC-06
Actor(s)	Administrator
Description	Allows the administrator to remove or hide inappropriate review content.
Preconditions	At least one review exists in the system.
Main Flow	1. Admin opens the Reviews section of the dashboard. 2. Admin browses or searches reviews flagged as inappropriate. 3. Admin selects a review and chooses to hide or delete it. 4. System removes the review from public view and recalculates the book's average rating.
Alternate Flow	If the review was the book's only review, the system resets the displayed rating to "No ratings yet" (FR-30, FR-27).
Postcondition	Inappropriate content is no longer publicly visible; catalog quality is maintained (FR-30).

6. Non-Functional Requirements

6.1 Performance Requirements

ID	Requirement	Priority
NFR-01	The system shall load any core page within 3 seconds under normal load on a standard broadband connection.	High
NFR-02	The system shall support at least 50–100 concurrent users without noticeable performance degradation (adequate for a college demo/pilot).	Medium
NFR-03	Search and filter results shall be returned and displayed within 2 seconds of the query being submitted.	Medium

6.2 Security Requirements

ID	Requirement	Priority
NFR-04	Passwords shall be stored using a one-way hashing algorithm (e.g. bcrypt); plain-text passwords shall never be stored.	High
NFR-05	All API endpoints that access personal data or reading-list content shall require a valid	High

ID	Requirement	Priority
	authentication token.	
NFR-06	Role-based authorization shall ensure a Reader cannot access Admin-only endpoints.	High
NFR-07	All data in transit shall be encrypted using HTTPS/TLS.	High
NFR-08	User input (search queries, reviews, profile fields) shall be validated and sanitized on both client and server to prevent injection attacks.	High

6.3 Usability

- The interface shall be simple enough for a first-time reader to add a book to their reading list in under 4 clicks/taps.
- The system shall be fully responsive across desktop, tablet and mobile screen sizes.
- Error and success messages shall be clear, specific, and shown close to the relevant form field/action.

6.4 Reliability & Availability

- The system shall target 99% uptime during the demonstration/evaluation period.
- Reading-list and review data shall not be lost due to a single server restart (persisted in the database, not in memory).

6.5 Maintainability & Portability

- The codebase shall follow a modular MVC-style structure (routes/controllers/models on the backend, reusable components on the frontend) to ease future maintenance and extension by new student contributors.
- The system shall run on any OS capable of hosting Node.js and MongoDB (Windows, Linux, macOS), and shall be accessible from any modern browser without requiring a native app install.

6.6 Scalability

Although scoped for a single catalog/instance in the academic version, the data model and API design shall avoid hard-coded single-library assumptions where reasonably possible, so that multi-branch or multi-library support can be added without a full redesign.

7. Software Testing Considerations

Each functional requirement in Section 3 shall map to at least one test case. The table below gives representative examples to guide the team's test plan; the full test case document can be maintained separately (e.g. in a spreadsheet) using this format.

Test ID	Req.	Test Scenario	Expected Result
TC-01	FR-10	Search for a book by title keyword	Matching books are returned in the results list.
TC-02	FR-11	Apply a category and rating filter together	Only books matching both filters are shown.
TC-03	FR-19	Add a book to the reading list	Book appears under the selected status on the dashboard.
TC-04	FR-21	Change a reading-list entry's status	Book moves to the newly selected status tab.
TC-05	FR-06	Reader attempts to access Admin-only page	Access denied (403) / redirected to login.
TC-06	FR-02	Register with a weak password	Registration blocked with validation message.
TC-07	FR-25	Submit a rating and review for a book	Review saved; average rating updates immediately.
TC-08	FR-26	Edit an existing review	Updated text/rating is saved and reflected on the page.
TC-09	FR-28	Admin adds a new book with a duplicate ISBN	System rejects with a clear validation message.
TC-10	FR-29	Admin deletes a category still in use	System warns and requires confirmation/reassignment.
TC-11	FR-30	Admin removes an inappropriate review	Review no longer visible; rating recalculated.
TC-12	NFR-04	Inspect stored password in database	Password is stored as a bcrypt hash, never plain text.
TC-13	NFR-06	Reader attempts to call an Admin-only API endpoint	Request rejected with 403 Forbidden.
TC-14	NFR-01	Load the Catalog page on a broadband connection	Page renders within 3 seconds.
TC-15	FR-05	Request password reset with a registered email	OTP/reset link is emailed and successfully resets password.
TC-16	NFR-08	Submit a review containing script/HTML tags	Input is sanitized; no script execution or injection occurs.

Recommended testing types: Unit testing (backend controllers/validators using a framework such as Jest/Mocha), Integration testing (API endpoints with Postman/Jest+Supertest), and manual UI/UX testing across at least two browsers and one mobile viewport before final submission.

8. Proposed System Architecture

Given the team's familiarity and the project's realistic academic scope, a MERN stack (MongoDB, Express.js, React.js, Node.js) three-tier architecture is recommended for the Online Library Management System. This keeps the project fully JavaScript-based end-to-end, is well documented, free to host on student-friendly free tiers, and maps cleanly onto the module breakdown in Section 3.

Layer	Responsibility
Presentation Layer	React.js SPA – Reader / Admin UI, communicating over HTTPS / REST (JSON)
Application Layer	Node.js + Express.js REST API – Auth, Catalog, Reading List, Reviews business logic via Mongoose ODM
Data Layer	MongoDB – Users, Books, Categories, ReadingListItems, Reviews

Figure 8.1: Three-tier architecture of the Online Library Management System.

8.1 Suggested Technology Stack

Layer	Technology
Frontend	React.js, React Router, Axios, Bootstrap / Tailwind CSS
Backend	Node.js, Express.js, JWT, bcrypt, Nodemailer
Database	MongoDB with Mongoose
Version Control	Git & GitHub
Testing	Postman (API testing), Jest (unit testing)
Deployment (suggested)	Vercel/Netlify (frontend), Render/Railway (backend), MongoDB Atlas (database)

9. Data Model Overview (for Database Planning)

The following core entities and relationships are proposed as a starting point for schema design. This is intentionally kept minimal and realistic for a student project; fields can be extended during implementation.

9.1 Core Entities

User (base account)

Field	Type	Description
_id	ObjectId	Primary key, auto-generated.
name	String	Full name of the user.

Field	Type	Description
email	String	Unique login identifier.
passwordHash	String	Bcrypt-hashed password; never stored in plain text.
role	Enum	reader / admin.
profilePhotoUrl	String	Optional profile picture URL.
createdAt	Date	Account creation timestamp.

Book

Field	Type	Description
_id	ObjectId	Primary key.
title	String	Book title.
author	String	Author name(s).
categoryId	ObjectId (FK)	References Category._id.
description	String	Synopsis / summary shown on the details page.
isbn	String	Unique book identifier.
publisher	String	Publishing house.
publishedYear	Number	Year of publication.
coverImageUrl	String	Cover image URL.
averageRating	Number	Computed average of all reviews (denormalized).
createdAt	Date	Record creation timestamp.

Category

Field	Type	Description
_id	ObjectId	Primary key.
name	String	e.g. Fiction, Science, Biography.
description	String	Short description shown to readers.

ReadingListItem

Field	Type	Description
_id	ObjectId	Primary key.
readerId	ObjectId (FK)	References User._id.
bookId	ObjectId (FK)	References Book._id.
status	Enum	wantToRead / currentlyReading / completed.
isFavorite	Boolean	Whether the reader marked this entry as a favorite.
addedAt	Date	Timestamp the book was added to the list.

Review

Field	Type	Description
_id	ObjectId	Primary key.
bookId	ObjectId (FK)	References Book._id.
readerId	ObjectId (FK)	References User._id.
rating	Number	Star rating, 1-5.
comment	String	Written review text.
isHidden	Boolean	Set true when moderated/hidden by an Admin.
createdAt	Date	Timestamp the review was posted.

Notification

Field	Type	Description
_id	ObjectId	Primary key.
userId	ObjectId (FK)	References User._id (recipient).
message	String	Notification text.
type	Enum	registration / passwordReset / announcement.
isRead	Boolean	Whether the user has viewed it.
createdAt	Date	Timestamp generated.

9.2 Key Relationships

- A Category has many Books (one-to-many).
- A User (Reader) has many ReadingListItems; a Book appears in many ReadingListItems (many-to-many via ReadingListItem).
- A Book has many Reviews; a User (Reader) writes many Reviews (many-to-many via Review, typically one review per reader per book).
- A User can receive many Notifications (one-to-many).

10. Project Timeline & Work Breakdown

The table below gives an indicative week-by-week plan for a single-semester college team project. Actual pacing may be adjusted based on team size and evaluation milestones set by the department.

Phase	Weeks	Key Activities	Deliverable
Requirements & Design	1–2	Finalize scope, this SRS, wireframes, DB schema	Approved SRS (this document)
Core Backend Setup	3–4	Auth, User/Book/Category models, REST API skeleton	Working auth + base APIs
Catalog & Search	5–6	Browse, search, filter, book details page (Section 3.2, 3.3)	Functional browse & search flow
Reading List & Reviews	7–8	Reading list dashboard, ratings/reviews (Section 3.4, 3.5)	Functional reading-list & review features
Admin Portal & Polish	9	Book/category management, moderation, UI polish (Section 3.6)	Functional admin dashboard
Testing	10–11	Unit, integration and manual testing against Section 7	Test report, bug fixes
Documentation & Demo Prep	12	Finalize SRS/report, prepare demo data and presentation	Final submission package

11. Requirements Traceability Matrix

This matrix maps every functional requirement to its owning module, a representative test case, and the primary screen where it is exercised, supporting full requirement-to-test coverage for evaluation.

FR ID	Module	Test Coverage	Primary Screen
FR-01	User Registration & Authentication	Registration flow test	Landing / Register screen
FR-02	User Registration & Authentication	TC-06	Register screen

FR ID	Module	Test Coverage	Primary Screen
FR-03	User Registration & Authentication	Login flow test	Login screen
FR-04	User Registration & Authentication	Session/token test	All authenticated screens
FR-05	User Registration & Authentication	TC-15	Forgot Password screen
FR-06	User Registration & Authentication	TC-05	Role-restricted routes
FR-07	User Registration & Authentication	Logout test	All authenticated screens
FR-08	User Registration & Authentication	Profile update test	Reader Profile screen
FR-09	Book Catalog & Search	Category browse test	Catalog / Browse screen
FR-10	Book Catalog & Search	TC-01	Catalog / Browse screen
FR-11	Book Catalog & Search	TC-02	Catalog / Browse screen
FR-12	Book Catalog & Search	Pagination test	Catalog / Browse screen
FR-13	Book Catalog & Search	Sort order test	Catalog / Browse screen
FR-14	Book Catalog & Search	Guest browsing test	Landing / Catalog screen
FR-15	Book Details	Details rendering test	Book Details screen
FR-16	Book Details	Average rating display test	Book Details screen
FR-17	Book Details	Review list display test	Book Details screen
FR-18	Book Details	Add-to-list button test	Book Details screen
FR-19	Personal Reading List	TC-03	My Reading List screen
FR-20	Personal Reading List	Status categorization test	My Reading List screen
FR-21	Personal Reading List	TC-04	My Reading List screen
FR-22	Personal Reading List	Remove entry test	My Reading List screen
FR-23	Personal Reading List	Favorite toggle test	My Reading List screen
FR-24	Personal Reading List	Dashboard summary test	My Reading List screen
FR-25	Reviews & Ratings	TC-07	Book Details screen
FR-26	Reviews & Ratings	TC-08	Book Details screen
FR-27	Reviews & Ratings	Average recompute test	Book Details screen
FR-28	Administrator Portal	TC-09	Admin – Manage Books screen
FR-29	Administrator Portal	TC-10	Admin – Manage Categories screen

FR ID	Module	Test Coverage	Primary Screen
FR-30	Administrator Portal	TC-11	Admin – Reviews screen
FR-31	Administrator Portal	User management test	Admin – Readers screen
FR-32	Administrator Portal	Reports test	Admin – Reports screen
FR-33	Administrator Portal	Announcement test	Admin – Announcements screen
FR-34	Notifications	Registration email test	Email (external)
FR-35	Notifications	Password reset email test	Email (external)
FR-36	Notifications	In-app notice test	Landing / Dashboard screen

12. Risk Analysis

The following risks were identified for the project timeline and technical implementation, along with mitigation strategies to be reviewed periodically by the team and project guide.

Risk	Likelihood	Impact	Mitigation Strategy
Inconsistent or duplicate book records entered by Admin	Medium	Medium	Enforce a unique index on ISBN and validate required fields on the Add/Edit Book form.
Team member unavailability close to deadline	Medium	High	Maintain clear module ownership and weekly check-ins per Section 10 timeline.
SMTP/email service rate limits or downtime	Medium	Medium	Use a free-tier provider with generous limits (e.g. SendGrid) and log failed sends for retry.
Scope creep beyond Section 1.4 boundaries	High	Medium	Treat Section 13 (Future Enhancements) as strictly out of scope until core features are stable.
Underestimated testing time before submission	Medium	High	Start testing incrementally per module (Section 7) rather than only at the end.
Free-tier hosting cold-starts affecting demo performance	Medium	Low	Warm up the deployed server shortly before any live demonstration.
Inconsistent UI across modules built by different members	Medium	Low	Agree on a shared component library and colour palette early (Section 4.1).
Loss of local work due to no version control discipline	Low	High	Enforce regular commits/pushes to a shared Git repository (Section 8.1).

13. Future Enhancements (Out of Current Scope)

- Physical book issue/return (circulation) tracking with due dates and fines.

- Integration with an external book-metadata API (e.g. Google Books / Open Library) for auto-fill and ISBN lookup.
- Personalized book recommendations based on reading history.
- E-book preview/reading or downloadable sample chapters.
- Social features: following other readers, sharing reading lists.
- Mobile app (React Native) version of the platform.
- Barcode/ISBN scanning for faster catalog entry by Admin.

14. Glossary

Term	Definition
SRS	Software Requirements Specification
API	Application Programming Interface
REST	Representational State Transfer – a web API architecture style
JWT	JSON Web Token – used for stateless authentication
MERN	MongoDB, Express.js, React.js, Node.js – a full JavaScript web stack
CRUD	Create, Read, Update, Delete – basic data operations
ISBN	International Standard Book Number – a unique book identifier
OTP	One-Time Password
UI/UX	User Interface / User Experience

End of Document – Online Library Management System SRS v1.0